

Universidad Nacional del Sur

Arquitectura y Diseño de Sistemas — 2026

Polizing

Diagramas de Arquitectura

Documentación del Proyecto

Práctica de Arquitectura y Diseño de Sistemas 2026

Grupo 6

David Felder · Manuel Ducos · Gonzalo Ferraro · Luca Fueyo · Marcos Kravetz · Agustín Echavarría

Fecha: 27/05/2026

1. Introducción y Descripción del Sistema

Polizing es un sistema de administración de seguros pensado para productores y administrativos que trabajan vendiendo y gestionando pólizas de múltiples compañías aseguradoras. Su objetivo central es centralizar en una única herramienta toda la operación diaria del productor: alta y mantenimiento de clientes, contratación y renovación de pólizas, recepción de comprobantes de pago, registro de siniestros con documentación adjunta y consulta de la información histórica de cada póliza.

Hoy el productor de seguros trabaja con datos dispersos entre planillas, los portales propios de cada aseguradora (Federación Patronal, Berkley, etc.) y mensajes sueltos de WhatsApp con sus clientes. Polizing reemplaza ese mosaico por un **panel web único** para el productor y un **canal conversacional unificado por WhatsApp** para el cliente final, que ya usa esa app a diario y que no necesita registrarse en ninguna interfaz nueva.

El valor concreto que aporta el sistema es doble. Para el productor: menos tiempo cargando datos a mano, menos errores de tipeo, y trazabilidad completa de quién hizo qué y cuándo. Para el cliente final: poder pedir su tarjeta de circulación, mandar el comprobante de pago o reportar un siniestro a las 3 de la mañana desde el celular, sin esperar a que se abra la oficina.

1.1 Actores del sistema

- **Productor / Administrativo** (usuario interno). Personal de la agencia. Es el único actor con acceso directo al panel web. Inicia sesión, da de alta clientes (corporativos y personas físicas), registra y renueva pólizas, valida o rechaza comprobantes de pago cargados por los clientes, gestiona siniestros (los recibe, los lee, los deriva a la aseguradora) y consulta reportes de vencimientos y contrataciones.
- **Cliente** (usuario externo). Persona física o representante de un cliente corporativo que contrató una póliza a través de la agencia. **No accede al sistema directamente**: interactúa exclusivamente vía WhatsApp con el chatbot. Consulta sus pólizas, descarga la tarjeta de circulación, envía comprobantes de pago y reporta siniestros con fotos y documentos.

1.2 Sistemas externos

- **WhatsApp Cloud API (Meta)** — canal bidireccional con el cliente final. El chatbot recibe mensajes entrantes por webhook y envía mensajes/templates/medios (PDF de tarjeta de circulación, confirmaciones) a través de la API REST oficial de Meta. Es el único canal de comunicación con el cliente.
- **API Federación Patronal (REST)** — fuente de datos de la aseguradora. Polizing consulta y carga datos de pólizas emitidas en Federación Patronal. Flujo: Polizing llama al externo.
- **API Berkley (SOAP)** — equivalente a la anterior pero para la aseguradora Berkley. Mismo flujo (Polizing llama al externo) pero sobre protocolo SOAP, lo que obliga a una capa adaptadora distinta a la de Federación Patronal.
- **API de IA / Procesamiento de documentos** — servicio externo que recibe imágenes y PDFs cargados por el cliente (comprobantes de pago, fotos de siniestros, actas policiales) y devuelve los datos extraídos en formato estructurado. Flujo: Polizing/Chatbot le envía los archivos crudos y recibe el resultado del procesamiento.

1.3 Requerimientos funcionales y no funcionales

Requerimientos funcionales (derivados de los casos de uso, sección 1.4):

- **Gestión de clientes:** registrar, consultar, modificar y dar de baja clientes (corporativos y no corporativos); listar clientes.
- **Gestión de aseguradoras:** registrar y consultar empresas aseguradoras.
- **Gestión de pólizas:** registrar póliza, consultar póliza, registrar renovación, anular póliza.
- **Gestión de siniestros:** registrar datos del siniestro, adjuntar documentación al siniestro.
- **Gestión de pagos:** registrar pago de póliza; recuperar resumen de contrataciones.
- **Notificaciones automáticas:** notificar vencimiento de póliza, notificar realización de pago.
- **Autenticación y usuarios:** iniciar sesión, cerrar sesión, registrar usuario interno (productor/administrativo).

Requerimientos no funcionales (Entrega 1):

- **RNF-01 — Seguridad (Confidencialidad e Integridad):** el sistema maneja datos sensibles (DNI, CUIT, documentos personales, bienes asegurados). Es crítico garantizar que cada cliente solo acceda a la información de las pólizas que le corresponden, y que ningún tercero pueda interceptar o alterar esos datos.
- **RNF-02 — Interoperabilidad:** el núcleo del negocio del productor es operar con varias aseguradoras a la vez. La arquitectura debe estar preparada desde su raíz para integrarse de forma fluida con APIs heterogéneas (REST, SOAP) y agregar nuevas aseguradoras sin reescribir el núcleo.
- **RNF-03 — Usabilidad:** los flujos del panel deben minimizar la cantidad de clicks y errores de carga, mostrando vencimientos y estados de manera clara para que el productor pueda priorizar su día.
- **RNF-04 — Disponibilidad 24/7:** los siniestros no tienen horario. El sistema debe seguir respondiendo a consultas y a reportes de siniestros aunque sea fuera del horario de oficina o durante una ventana de mantenimiento de un componente.
- **RNF-05 — Mantenibilidad:** el mercado asegurador cambia (nuevas normativas, nuevas coberturas, nuevas integraciones). El código debe ser modular para que el equipo pueda evolucionar el sistema sin romper lo existente.
- **RNF-06 — Trazabilidad:** cada cambio sobre datos críticos (pólizas, siniestros, pagos) debe quedar registrado con su autor y su fecha. La trazabilidad protege legal y comercialmente al productor.

1.4 Casos de Uso – Historias de Usuarios

Los casos de uso completos están en [Entregables/Entrega 1/Casos de uso/](#). El diagrama general se encuentra en [Entregables/Entrega 1/Casos de uso/Diagrama de casos de uso/](#). La lista de casos de uso identificados es:

ID	Caso de uso
CU-10	Registrar Cliente
CU-11	Consultar Cliente
CU-12	Modificar datos de Cliente
CU-13	Dar de baja Cliente
CU-20	Registrar Empresa de Seguro
CU-21	Recuperar / Consultar empresa de seguro
CU-22	Registrar Póliza
CU-23	Consultar Póliza
CU-24	Registrar Renovación de Póliza
CU-25	Anular Póliza
CU-30	Registrar datos de Siniestro
CU-31	Adjuntar documentación de Siniestro
CU-40	Recuperar resumen de contrataciones
CU-41	Registrar pago de póliza
CU-50	Notificar vencimiento de póliza
CU-51	Notificar realización de pago
CU-60	Iniciar Sesión
CU-61	Cerrar Sesión
CU-62	Registrar Usuario
CU-70	Listar clientes

Todos los casos fueron actualizados según las devoluciones del corrector (ver `Entregables/Entrega 3/correcciones-grupo-6.txt`).

1.5 Modelo de Entidad Relación o Diagrama de Clases

La última versión depurada del modelo de datos vive en `Entregables/Entrega 3/erd.svg` (diagrama) y `Entregables/Entrega 3/modelo-datos.md` (descripción tabla por tabla). Pasó de 17 tablas iniciales a **10 tablas** finales, eliminando 7 tablas puente innecesarias y aplicando 3NF, enums para valores cerrados y auditoría por actor.

Tablas finales: `clientes` , `clientes_corporativos` , `clientes_no_corporativos` , `empresas_aseguradoras` , `tipos_seguro` , `polizas` , `siniestros` , `siniestro_documentos` , `pagos` , `usuarios` .

1.6 Modelo Conceptual de Datos consistente con el Modelo de Entidad Relación o con el Diagrama de Clases, según corresponda

El modelo conceptual coincide con la jerarquía de conceptos definida en el Glosario (Entrega 1) y materializada en el ERD:

- Un **Cliente** (corporativo o no corporativo, vía discriminador `tipo`) contrata una o más **Pólizas**.
- Cada **Póliza** se emite contra una **Empresa Aseguradora** y se clasifica por un **Tipo de Seguro** y un grado de **Cobertura** (enum: `responsabilidad_civil` | `terceros_completo` | `todo_riesgo` | `basica` | `integral`).
- Si ocurre un evento cubierto, se registra un **Siniestro** vinculado a esa póliza, con N **Documentos** adjuntos (imágenes o PDFs).
- Para mantener vigentes sus pólizas, el cliente emite un **Pago** que puede agrupar varias pólizas en un mismo batch (relación 1 pago : N pólizas, materializada como `polizas.pago_id`).
- Los **Usuarios** internos (productor / administrativo) operan el sistema y quedan registrados como actores en los campos de auditoría (`modificado_por_id` + `modificado_en`) de las entidades críticas.

2. Diagramas de Arquitectura

La arquitectura del sistema está documentada con el modelo **C4** en tres niveles. Los diagramas viven en `Entregables/Entrega 2/contexto-container-componentes.pdf` (PDF consolidado) y, abiertos por separado, en `Entregables/Entrega 2/Diagrama de Contexto/`, `Entregables/Entrega 2/Diagrama De Contenedores/` y `Entregables/Entrega 2/Diagrama de Componentes/`.

Entregables producidos:

- **Diagrama de Contexto (C4 Nivel 1):** define los límites del sistema y sus interacciones con actores y sistemas externos.
- **Diagrama de Contenedores (C4 Nivel 2):** especifica la arquitectura distribuida con frontend, backend, base de datos y servicios satélite.
- **Diagrama de Componentes (C4 Nivel 3):** detalla la separación de responsabilidades interna del contenedor de backend.

2.1. Diagrama de Contexto

El diagrama de contexto representa a Polizing como una caja negra que conversa con dos actores humanos y tres sistemas externos. No expone detalles internos: solo el qué entra y qué sale del sistema.

2.1.1 Flujos principales identificados

- **Flujo 1:** Productor/Administrativo ↔ Polizing — el productor inicia sesión y gestiona pólizas, clientes y reportes; registra los datos pertinentes y atiende eventos que requieren acción humana (siniestros nuevos, pagos a validar).
- **Flujo 2:** Polizing → Cliente — Polizing envía al cliente notificaciones sobre operaciones que le incumben (vencimiento de póliza, confirmación de pago registrado).
- **Flujo 3:** Polizing ↔ Sistema de Chatbot — Polizing y el Chatbot intercambian datos a través de un canal interno; el Chatbot actúa como intermediario entre el cliente y el sistema principal.
- **Flujo 4:** Cliente ↔ Sistema de Chatbot (vía WhatsApp) — el cliente consulta sus pólizas, vencimientos, registra siniestros y envía comprobantes; el chatbot interpreta la información y responde.
- **Flujo 5:** Polizing → API Federación Patronal (REST) — Polizing consulta y carga datos de las pólizas de Federación Patronal.

- **Flujo 6:** Polizing → API Berkley (SOAP) — equivalente al flujo 5 pero contra Berkley sobre SOAP.

2.1.2 Decisiones de diseño en el nivel de contexto

- **Decisión 1 — El Chatbot queda fuera del sistema "Polizing" como un sistema de software independiente.** Si bien funcionalmente forman un todo desde el punto de vista del usuario, técnicamente son dos sistemas distintos con ciclos de release independientes y stacks distintos. Esto permite que un cambio en el bot no requiera redespargar el panel, y viceversa. Esta decisión se profundiza y justifica en ADR-001 (Entrega 4).
- **Decisión 2 — Las APIs de las aseguradoras quedan fuera del límite del sistema y se consumen como sistemas externos.** No se replica internamente la lógica de cada aseguradora; Polizing las consulta cuando necesita datos canónicos y guarda copia local de lo que el productor opera. Esto evita duplicar la fuente de verdad de cada aseguradora y permite agregar nuevas integraciones sin tocar el núcleo.
- **Decisión 3 — Los clientes no son usuarios del sistema principal.** Se delega toda la interacción con el cliente final al Chatbot vía WhatsApp. No se construye una app móvil ni un portal web para el cliente: ya usa WhatsApp todos los días, y un canal extra bajaría la adopción.

2.2 Diagrama de Contenedores

El diagrama de contenedores descompone el sistema en unidades desplegables independientes. Cada contenedor representa una tecnología concreta y una responsabilidad bien delimitada.

2.2.1 Contenedores del sistema

Frontend — Next.js (React) Panel de control web que usan productores y administrativos. Permite iniciar sesión, listar clientes, registrar pólizas, ver siniestros y validar comprobantes. Se comunica con el Backend mediante HTTPS (Server Actions y endpoints REST internos). Vive en el directorio `my-app/`.

Backend — Next.js (App Router + Server Actions + API Routes) Contiene toda la lógica de negocio del panel. Expone una API REST que consume tanto el Frontend (vía Server Actions) como el Chatbot (vía HTTP server-to-server). Recibe los webhooks de cambio de estado del Chatbot (ej. nuevo siniestro creado por el cliente) y dispara las notificaciones programadas (vencimientos de pólizas). Vive en el mismo directorio `my-app/` que el Frontend, integrados por ser una aplicación Next.js fullstack.

Base de Datos — PostgreSQL (administrada por Supabase) Almacena el estado del sistema: clientes, aseguradoras, pólizas, siniestros, documentos, pagos y usuarios internos. Se accede mediante Prisma ORM desde el Backend usando SQL. Soporta transacciones ACID multi-tabla y joins complejos para los reportes.

Sistema de Chatbot — FastAPI (Python) Servicio independiente que conversa con el cliente final por WhatsApp. Expone un webhook público que Meta llama cada vez que un cliente escribe; mantiene el estado conversacional (flujo y paso actual) en una base de datos SQLite local; y consume la API REST del Backend para validar al cliente, listar pólizas, registrar siniestros y enviar comprobantes. Vive en el directorio `chatbot/`.

API WhatsApp Cloud — JSON/HTTPS Componente externo (Meta) que conecta al Chatbot con la red de mensajería WhatsApp. Recibe mensajes salientes del bot y le entrega los entrantes vía webhook firmado.

API IA — JSON/HTTPS (sistema satélite, integración planificada) Servicio de procesamiento de documentos que recibe imágenes y PDFs (comprobantes, fotos de siniestros) y devuelve los datos extraídos estructurados para que el Backend pueda persistirlos. Hoy se invoca desde el Chatbot al recibir adjuntos del cliente.

APIs de Aseguradoras — Federación Patronal (REST) y Berkley (SOAP) Sistemas externos de cada compañía. El Backend de Polizing las consulta para sincronizar datos canónicos de pólizas.

2.2.2 Decisiones arquitectónicas

Las decisiones de esta sección están registradas en detalle como ADRs en `Entregables/Entrega 4/ADRs/`. Aquí se resumen las justificaciones clave.

Organización de servicios

Se adopta una división en **dos servicios desplegables independientes** (ADR-001): el panel web (`my-app` , monolito modular Next.js) y el chatbot (`chatbot` , servicio FastAPI). No es monolito único ni microservicios completos. La división responde a tres motivadores: (a) dos audiencias con perfiles muy distintos (productor en web vs cliente en WhatsApp), (b) requerimientos de runtime opuestos (serverless funciona para el panel pero rompe el webhook del bot, que necesita proceso persistente y estado local), (c) ritmos de evolución distintos (un cambio de copy del bot no debe forzar un deploy del panel). Las limitaciones asumidas conscientemente: el modelo de dominio se duplica parcialmente entre los dos servicios y hay dos pipelines de CI/CD.

Estrategia de persistencia

Se elige **PostgreSQL relacional** (administrado por Supabase) como base única del panel (ADR-002): el modelo de dominio es fuertemente relacional (clientes ↔ pólizas ↔ siniestros ↔ pagos), las transacciones multi-tabla son frecuentes (alta de siniestro + documentos, validación de pago) y los reportes requieren joins complejos. NoSQL fue descartado porque obligaría a emular joins en código aplicativo. El chatbot mantiene su propia **SQLite local** solo para el estado conversacional (qué flujo y paso lleva cada teléfono); los datos de negocio nunca se duplican en el bot, se consultan vía API al panel. El criterio que define qué datos van a cada base es claro: datos de negocio en Postgres, estado conversacional efímero en SQLite local.

Mecanismo de comunicación entre componentes

La comunicación entre el chatbot y el panel es **REST síncrono server-to-server sobre HTTPS**, autenticada con header `X-API-Key` (ADR-003). Las alternativas asíncronas (colas de mensajes, eventos pub-sub) se descartaron por dos razones: el patrón de uso del bot es estrictamente petición/respuesta (cada mensaje del cliente exige una respuesta inmediata), y el volumen acotado por la cadencia humana no justifica el overhead de un broker. Dentro del panel, el Frontend habla con el Backend mediante Server Actions y endpoints HTTP internos. El trade-off asumido: si el panel cae, el bot no puede responder al cliente; no hay desacoplamiento temporal.

2.3. Diagrama de Componentes

El diagrama de componentes detalla la estructura interna del contenedor Backend de Polizing. Los componentes externos al backend (Frontend, DataBase, Chatbot, APIs de aseguradoras) se incluyen como cajas grises de referencia.

2.3.1 Arquitectura interna adoptada

El Backend está organizado en **capas por responsabilidad**, con dos cortes:

- **Corte vertical por dominio:** Pólizas, Clientes, Siniestros, Aseguradoras y Pagos son áreas de negocio independientes con sus propios controladores y repositorios. Esto refleja el alcance del producto y facilita que distintos integrantes del equipo trabajen en paralelo sin colisionar.
- **Corte horizontal por capa:** Controladores de negocio (Poliza Controller, Client Management, etc.), Componentes transversales (Login/Logout, Security, Reset Password, Notification Controller, WhatsApp Webhook Controller), y Capa de acceso a datos (Data Controllers Pólizas, Clientes, Siniestros) que encapsulan las consultas SQL contra Postgres.

El criterio de separación es mantener cada componente con una sola responsabilidad clara y comunicaciones explícitas: los Controladores de negocio orquestan lógica y delegan persistencia en los Data Controllers; los componentes transversales (Security, Notifications) son consumidos por los anteriores pero no consumen lógica de negocio.

2.3.2 Descripción de componentes

Componentes de autenticación y seguridad

- **Login / Logout** — recibe credenciales del frontend, las valida contra el componente Security y emite/destruye la sesión del usuario interno.
- **Reset Password** — gestiona el flujo de restablecimiento de contraseña del usuario interno.
- **Security** — componente transversal que valida credenciales y gestiona los roles de acceso (Productor, Administrativo). Es consultado por Login/Logout, Reset Password y por cada controlador de negocio antes de exponer una operación protegida.

Controladores de negocio

- **Poliza Controller** — gestiona la creación, consulta y seguimiento de pólizas; registra siniestros y sus documentos adjuntos. Consulta a Client Management para validar el cliente dueño de la póliza y delega persistencia en Data Controller Pólizas y Data Controller Siniestros.
- **Client Management** — administra el ciclo de vida de los clientes (alta, modificación, baja, consulta) y su información asociada. Delega en Data Controller Clientes.
- **Notification Controller** — gestiona recordatorios de vencimientos, confirmaciones de pago, nuevas pólizas y notificaciones de siniestros. Usa templates para despachar por WhatsApp (vía API Chatbot) o por email. Es invocado por el Timer interno del backend para los vencimientos programados.
- **WhatsApp Webhook Controller** — recibe y procesa los webhooks entrantes desde la API de WhatsApp Cloud (mensajes, adjuntos, comprobantes enviados por los clientes). Es el punto de entrada de toda la comunicación inbound del canal cliente.

Capa de acceso a datos

- **Data Controller Pólizas** — encapsula las consultas SQL, escrituras y lecturas hacia la base PostgreSQL mapeando las pólizas. Único componente que conoce el schema de la tabla `polizas`.
- **Data Controller Clientes** — equivalente al anterior para clientes (corporativos y no corporativos) y sus pagos asociados.
- **Data Controller Siniestros** — equivalente para siniestros y sus documentos adjuntos.

3. Stack Tecnológico

#	Tecnología	Uso en el sistema	Justificación
1	Next.js 16 (App Router) + TypeScript	Framework del panel web (my-app): rutas, Server Components, Server Actions, API Routes.	Fullstack en un solo proyecto, server components reducen JS en cliente, Server Actions simplifican mutations sin construir REST a mano. TypeScript da seguridad de tipos en compile-time.
2	React 19 + shadcn/ui + Tailwind CSS	UI del panel: componentes reutilizables, formularios, listados, tablas.	shadcn/ui da componentes accesibles y personalizables;

			Tailwind acelera el estilado consistente sin CSS suelto.
3	PostgreSQL (administrado por Supabase)	Base de datos principal del panel.	Modelo de dominio relacional con transacciones ACID y joins complejos (ADR-002). Supabase elimina la operación del motor y aporta connection pooling.
4	Prisma ORM	Capa de acceso a datos del panel: schema declarativo, migraciones, queries tipadas.	Genera tipos TypeScript desde el schema, elimina una clase entera de errores de runtime y es la herramienta con la que el equipo ya tiene experiencia.
5	Zod + React Hook Form	Validación de formularios y payloads en my-app.	Validación unificada entre cliente y servidor con una sola definición de schema.
6	Python 3.13 + FastAPI	Servicio Chatbot (chatbot/): orquestación de conversaciones, webhook de WhatsApp.	FastAPI tiene runtime async ideal para webhooks y clientes HTTP; el ecosistema Python concentra las librerías de WhatsApp y de procesamiento de documentos.
7	SQLAlchemy 2.0 + SQLite	Persistencia local del estado conversacional del chatbot.	SQLite es cero-configuración para el estado conversacional acotado del bot; el modelo de negocio vive en Postgres del panel (no se duplica).
8	httpx (async)	Cliente HTTP en el chatbot para llamar a Meta y al backend del panel.	Cliente async moderno, encaja con FastAPI; reemplaza requests para correr sin bloquear el event loop.
9	WhatsApp Cloud API (Meta)	Canal con el cliente final: webhook entrante, envío de mensajes/templates/medios.	Acceso oficial directo al canal, sin markup de agregadores, con tier gratuito suficiente para el PoC (ADR-005).
10	Vercel	Hosting del panel my-app.	Integración nativa con Next.js, preview deploys por PR, TLS automático, encaja con Supabase (ADR-004).
11	PaaS de contenedores (Fly.io / Railway)	Hosting del chatbot chatbot/.	Permite correr un proceso persistente con volumen para SQLite, expone URL pública estable que Meta requiere como webhook (ADR-004).

12	Autenticación custom con cookies httpOnly	Sesión del usuario interno en my-app.	Stack acotado de roles (productor / administrativo), sin necesidad de OAuth de terceros; el dominio no exige federación de identidades.
----	---	---------------------------------------	---

4. Patrones de Diseño y Atributos de Calidad

4.1 Patrones aplicados

- **Repository Pattern (Data Controllers)** — los componentes `Data Controller` `Pólizas`, `Data Controller Clientes` y `Data Controller Siniestros` aíslan a los controladores de negocio del schema concreto y del motor SQL. Si mañana se reescribe una query, solo se toca un archivo.
- **Adapter / Anti-Corruption Layer (capa de aseguradoras y MainSystemClient)** — la integración con Federación Patronal (REST) y Berkley (SOAP) se aísla en módulos adaptadores que traducen las respuestas heterogéneas al modelo interno de Polizing. En el chatbot, `chatbot/app/main_system_client.py` aplica el mismo patrón para hablar con el panel y soporta un modo `mock` para desarrollo y tests.
- **Strategy / Polymorphic Handlers (intents del chatbot)** — los flujos conversacionales (`CirculationCardHandler`, `PaymentReceiptHandler`, `ClaimHandler`) heredan de un `BaseFlowHandler` común. El motor conversacional selecciona el handler según el flujo activo del usuario, sin if/else extensos.
- **State Machine (motor conversacional)** — la tabla `conversations` mantiene `current_flow` y `current_step` por teléfono; el `ConversationEngine` los lee, decide qué responder y actualiza el estado. Es determinístico, debuggable y no depende de un LLM.
- **Webhook / Observer (WhatsApp Webhook Controller y notificaciones)** — el sistema reacciona a eventos externos (mensajes de Meta) y emite eventos salientes (notificaciones de vencimiento) usando webhooks y handlers desacoplados del flujo de request del panel.
- **Singleton (cliente Prisma)** — `my-app/lib/prisma.ts` expone una única instancia del cliente, evitando agotar el pool de conexiones de Postgres en entornos serverless con muchos invocadores concurrentes.

4.2 Atributos de calidad

RNF	Cómo lo satisface la arquitectura
RNF-01 Seguridad (Confidencialidad e Integridad)	Comunicación toda sobre HTTPS (Vercel + PaaS de contenedores proveen TLS automático). Autenticación del usuario interno por cookie httpOnly. Comunicación server-to-server entre chatbot y panel autenticada con X-API-Key (ADR-003). Verificación HMAC del webhook de WhatsApp contra APP_SECRET. Datos en Postgres administrado (Supabase aplica encriptación en reposo y reglas de red). El componente Security centraliza la validación de roles.
RNF-02 Interoperabilidad	Las integraciones externas (Federación Patronal REST, Berkley SOAP, API IA, WhatsApp Cloud API) están encapsuladas en módulos adaptadores específicos (ver patrón Anti-Corruption Layer). Agregar una nueva aseguradora consiste en escribir un nuevo adaptador sin tocar el núcleo del panel.
RNF-03 Usabilidad	Frontend Next.js + shadcn/ui ofrece componentes accesibles y consistentes. Server Components reducen el tiempo de primera pintura.

	Listados con índices compuestos en Postgres (ej. (estado, fecha_fin_vigencia) en pólizas) permiten que el productor vea vencimientos próximos sin lag.
RNF-04 Disponibilidad 24/7	División en dos servicios independientes (ADR-001): la caída del panel no afecta al canal cliente y viceversa. Vercel y el PaaS de contenedores proveen restart automático ante fallos. Postgres de Supabase tiene SLA y backups gestionados. El webhook del chatbot vive en un proceso persistente, no se pierde entre invocaciones serverless.
RNF-05 Mantenibilidad	Organización por capas y por dominio en el backend (Data Controllers + Controladores de negocio + componentes transversales). Schema de Postgres declarativo (Prisma) con migraciones versionadas. ADRs documentando cada decisión estructural permiten que un integrante nuevo entienda el por qué de cada elección. Patrones aplicados (Repository, Adapter, Strategy) aíslan cambios.
RNF-06 Trazabilidad	Modelo de datos con auditoría por actor en cada tabla de negocio: modificado_por_id (FK a usuarios) y modificado_en (timestamp) en clientes, pólizas, siniestros, pagos, empresas_aseguradoras. Cada cambio en datos críticos queda atribuido a un usuario y un momento exacto, soportando auditoría legal y comercial.

5. Flujos Principales del Sistema

5.1 Flujo de autenticación

1. El productor accede al panel `my-app` y completa el formulario de Login (componente Frontend).
2. El componente **Login / Logout** recibe las credenciales y delega su validación en el componente **Security**.
3. Security verifica las credenciales contra el **Data Controller Clientes** (que en este caso consulta la tabla `usuarios`) y valida el rol asignado (Productor o Administrativo).
4. Si las credenciales son válidas, Login emite una **cookie de sesión** `httpOnly (polizing-session)`, firmada y con TTL de 7 días.
5. Cada request posterior del productor lleva la cookie. Cada controlador de negocio (Póliza Controller, Client Management, etc.) consulta a Security antes de exponer la operación; Security la rechaza con 401 si la cookie es inválida o expiró.
6. Logout invalida la cookie en cliente y limpia cualquier rastro de sesión asociada en el servidor.

Notas: la API server-to-server expuesta al chatbot **no** usa esta cookie; usa el header `X-API-Key` validado en un middleware separado (ver ADR-003 y flujo 5.2).

5.2 Flujo de carga de un siniestro vía WhatsApp (flujo central del dominio)

Este es el flujo más representativo del sistema: ejercita los dos servicios, el canal externo (WhatsApp), la API IA y el modelo de datos completo (siniestro + documentos en una sola transacción).

1. El **Cliente** escribe al número de WhatsApp del productor: "tuve un choque".
2. **API WhatsApp Cloud** entrega el mensaje al **WhatsApp Webhook Controller** del Chatbot (vía webhook firmado, verificado con HMAC).
3. El **ConversationEngine** del chatbot busca la conversación del teléfono en la tabla local `conversations`. Como no hay un flujo activo, presenta el menú de opciones; el cliente elige "4 —

Reportar siniestro".

4. El motor crea/actualiza la conversación con `current_flow = claim, current_step = 0` y delega en el **ClaimHandler**.
5. Antes de continuar, el handler llama al panel: `GET /api/clients/by-phone/{phone}` con header `X-API-Key` . Si devuelve 404, el bot responde "no estás registrado" y corta. Si devuelve 200, sigue.
6. El handler pide la póliza: llama `GET /api/policies?phone={phone}` y le presenta al cliente la lista de pólizas. El cliente elige una.
7. El handler conduce al cliente paso a paso por el flujo: fecha, hora, lugar, descripción, datos de terceros, fotos, carnet, tarjeta verde, acta policial (opcional). Cada respuesta del cliente actualiza `conversations.data_json` y avanza `current_step` .
8. Cada adjunto se descarga de Meta vía `WhatsAppClient` y se envía al servicio de **API IA** para extracción y validación; el resultado estructurado se acumula en `data_json` .
9. Cuando el cliente envía "FINALIZAR", el handler arma el payload completo y hace `POST /api/claims` al panel.
10. En el panel, **Poliza Controller** valida el payload, llama a **Data Controller Siniestros** y abre una transacción Postgres: inserta el `siniestro` y los N registros en `siniestro_documentos` atómicamente. La transacción registra `modificado_por_id = NULL` (alta automática vía chatbot) y `modificado_en = now()` .
11. **Notification Controller** se entera del nuevo siniestro y dispara una notificación al productor (vía WhatsApp template o email) para que lo lea y lo derive.
12. El panel responde 201 al chatbot; el chatbot confirma al cliente: "Tu siniestro fue registrado con número SIN-00123. Te contactamos en cuanto un asesor lo revise."
13. El productor entra al panel, ve el siniestro nuevo con `leido = false` , lo marca como leído y avanza con el caso desde Poliza Controller.

5.3 Flujo de notificación de vencimiento de póliza (procesamiento batch)

Este flujo ejerce el componente Notification Controller y el canal saliente hacia el cliente vía Chatbot/WhatsApp.

1. Un **Timer interno** del Backend (cron o tarea programada) se ejecuta una vez por día.
2. El timer invoca al **Notification Controller**, que pide a **Data Controller Pólizas** las pólizas con `fecha_fin_vigencia` dentro de los próximos N días (por ejemplo, 7) y estado `vigente` . La consulta usa el índice compuesto (`estado, fecha_fin_vigencia`) .
3. Por cada póliza próxima a vencer, Notification Controller construye el mensaje a partir de una **plantilla aprobada** registrada en WhatsApp Business Manager y, mediante una llamada HTTPS al servicio de Chatbot (o directamente a la API de WhatsApp Cloud, según despliegue), lo envía al teléfono del cliente.
4. La **API WhatsApp Cloud** entrega el mensaje al cliente y devuelve el ID del envío.
5. Notification Controller registra el envío para evitar notificar dos veces la misma vigencia (idempotencia controlada por estado en BD).
6. Si el cliente responde al mensaje (ej. "quiero renovar"), el flujo entra como mensaje nuevo en el WhatsApp Webhook Controller y se procesa con el handler conversacional correspondiente — sin tocar el flujo batch.

6. Conclusión

La arquitectura de Polizing responde directamente al dominio del productor de seguros: un negocio con dos canales fundamentalmente distintos (panel interno de gestión + atención al cliente final por mensajería), integración constante con compañías externas heterogéneas (REST y SOAP) y operación 24/7 motivada por

la naturaleza de los siniestros. Por eso la decisión arquitectónica más importante —documentada en ADR-001— fue **dividir el sistema en dos servicios desplegados independientes** (`my-app` y `chatbot`) en lugar de adoptar un monolito único o una constelación de microservicios. Esta organización aísla los ritmos de evolución, los stacks tecnológicos y los modos de falla de cada audiencia, sin pagar el costo operativo de operar muchos servicios.

Sobre esa división se apoyan el resto de las decisiones: PostgreSQL relacional con Prisma para el panel — porque el modelo es naturalmente relacional y los reportes lo exigen— y SQLite local en el chatbot solo para el estado conversacional, sin duplicar el dominio de negocio. La comunicación entre ambos servicios es REST síncrono autenticado con API Key, una elección deliberadamente simple frente al overhead de colas o eventos que no se justifica en este volumen. El despliegue heterogéneo (Vercel para el panel + PaaS de contenedores para el chatbot) materializa la separación a nivel infraestructura.

Los **trade-offs asumidos conscientemente** son visibles y conocidos:

- El acoplamiento temporal entre chatbot y panel (sin colas intermedias) implica que si el panel cae, el bot no puede responder. Aceptado por simplicidad y porque el patrón conversacional es petición/respuesta.
- La duplicación parcial del modelo de dominio entre los dos servicios (el chatbot tiene su propia representación liviana de cliente, póliza, siniestro). Aceptado porque mantenerlos sincronizados por contrato HTTP es más simple que un esquema compartido.
- Dos pipelines de CI/CD, dos paneles de observabilidad y dos lugares donde rotar secretos. Aceptado para mantener cada servicio en la plataforma que mejor encaja con su runtime.
- La integración directa con Meta Cloud API (sin agregador como Twilio) implica gestionar templates y rate limits a mano. Aceptado por costo y por el volumen acotado del PoC.

Si el sistema creciera, los puntos a revisar serían: introducir una cola de mensajes para desacoplar el bot del panel y soportar reintentos automáticos; replicar Postgres con réplicas de lectura o sharding por aseguradora si el volumen lo exigiera; migrar de SQLite a un Redis o Postgres compartido para el estado conversacional si se requiriera escalar horizontalmente el chatbot; y eventualmente considerar un BSP (Business Solution Provider) para WhatsApp si la operación pasara de PoC a producción real con SLAs comerciales.